# BCA HUB ■

# C PROGRAMMING

Complete First-Year Study Notes

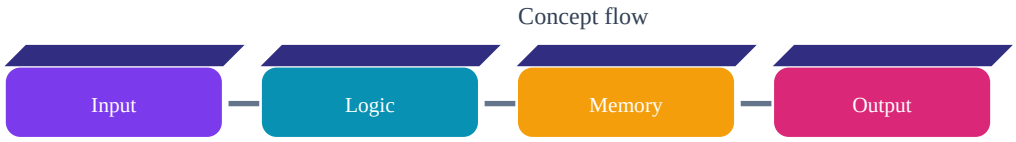
Concepts • Diagrams • Programs • Q&A; • MCQs • Viva

C programming workflow

# Contents

# 1. Introduction to C

C is a general-purpose procedural programming language. It is widely used for system software, embedded programming and learning core programming concepts.




## ■ Key Points

• Fast and efficient

• Portable

• Structured

• Rich operators

• Supports pointers

## ■ Example

```
#include
int main(void) {
printf("Hello, BCA!");
return 0;
}
```

## ■ Explanation

Read the code from top to bottom. Identify the input, processing logic, variables used, and final output. Then type the program yourself and change one value to see how the result changes.

## ■ Question: What is the main idea of this chapter?

■ **Answer:** Understand the concept, its syntax, and where it is useful in a C program.

## ■ Question: Why is this topic important?

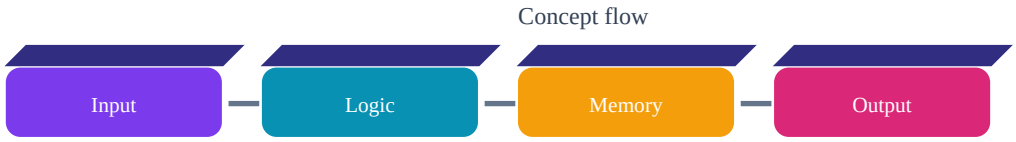
■ **Answer:** It is a foundation for writing, reading, debugging and explaining C programs.

## ■ Question: What should a beginner practice?

■ **Answer:** Write small programs, compile them, predict the output, and then run them.

# 2. Tokens, Keywords & Identifiers

Tokens are the smallest meaningful elements of a C program. They include keywords, identifiers, constants, strings, operators and punctuators.




## ■ Key Points

• int, if and return are keywords

• Identifiers are programmer-defined names

• Constants represent fixed values

• Strings use double quotes

• C is case-sensitive

## ■ Example

```
int marks = 85;
float percentage = 82.5f;
char grade = 'A';
```

## ■ Explanation

Read the code from top to bottom. Identify the input, processing logic, variables used, and final output. Then type the program yourself and change one value to see how the result changes.

## ■ Question: What is the main idea of this chapter?

■ **Answer:** Understand the concept, its syntax, and where it is useful in a C program.

## ■ Question: Why is this topic important?

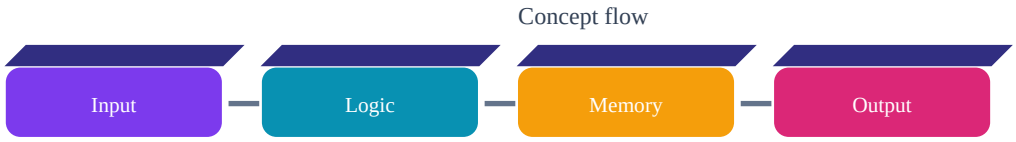
■ **Answer:** It is a foundation for writing, reading, debugging and explaining C programs.

## ■ Question: What should a beginner practice?

■ **Answer:** Write small programs, compile them, predict the output, and then run them.

# 3. Data Types, Variables & Constants

A data type tells the compiler what kind of value a variable can store. Variables are named storage locations.




## ■ Key Points

• char, int, float and double

• Declaration introduces a variable

• Initialization gives an initial value

• const can represent a value intended not to change

## ■ Example

```
int age = 18;
float marks = 91.5f;
char section = 'A';
const int DAYS = 7;
```

## ■ Explanation

Read the code from top to bottom. Identify the input, processing logic, variables used, and final output. Then type the program yourself and change one value to see how the result changes.

## ■ Question: What is the main idea of this chapter?

■ **Answer:** Understand the concept, its syntax, and where it is useful in a C program.

## ■ Question: Why is this topic important?

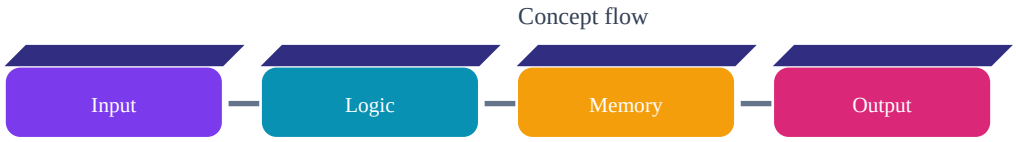
■ **Answer:** It is a foundation for writing, reading, debugging and explaining C programs.

## ■ Question: What should a beginner practice?

■ **Answer:** Write small programs, compile them, predict the output, and then run them.

# 4. Input & Output

The stdio.h library provides common input/output functions. printf displays formatted output and scanf reads formatted input.




## ■ Key Points

• printf() for output

• scanf() for formatted input

• %d = int, %f = float, %c = char, %s = string

• scanf commonly receives an address

## ■ Example

```
int n;
printf("Enter n: ");
scanf("%d", &n;);
printf("You entered %d", n);
```

## ■ Explanation

Read the code from top to bottom. Identify the input, processing logic, variables used, and final output. Then type the program yourself and change one value to see how the result changes.

## ■ Question: What is the main idea of this chapter?

■ **Answer:** Understand the concept, its syntax, and where it is useful in a C program.

## ■ Question: Why is this topic important?

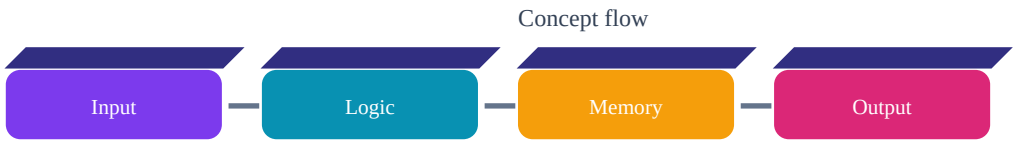
■ **Answer:** It is a foundation for writing, reading, debugging and explaining C programs.

## ■ Question: What should a beginner practice?

■ **Answer:** Write small programs, compile them, predict the output, and then run them.

# 5. Operators & Expressions

Operators perform operations on values. Expressions combine operands and operators to produce results.




## ■ Key Points

• Arithmetic: + - * / %

• Relational: < > <= >= == !=

• Logical: && || !

• Assignment: = += -= *= /=

• Increment/decrement: ++ --

## ■ Example

```
int a = 10, b = 3;
printf("%d", a + b);
printf("%d", a % b);
```

## ■ Explanation

Read the code from top to bottom. Identify the input, processing logic, variables used, and final output. Then type the program yourself and change one value to see how the result changes.

## ■ Question: What is the main idea of this chapter?

■ **Answer:** Understand the concept, its syntax, and where it is useful in a C program.

## ■ Question: Why is this topic important?

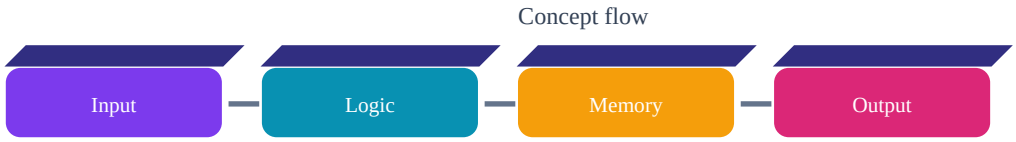
■ **Answer:** It is a foundation for writing, reading, debugging and explaining C programs.

## ■ Question: What should a beginner practice?

■ **Answer:** Write small programs, compile them, predict the output, and then run them.

# 6. Decision Making

Conditional statements allow a program to choose different execution paths.




## ■ Key Points

• if

• if-else

• else-if

• switch

• break

## ■ Example

```
int marks;
scanf("%d", &marks;);
if (marks >= 40)
printf("Pass");
else
printf("Fail");
```

## ■ Explanation

Read the code from top to bottom. Identify the input, processing logic, variables used, and final output. Then type the program yourself and change one value to see how the result changes.

## ■ Question: What is the main idea of this chapter?

■ **Answer:** Understand the concept, its syntax, and where it is useful in a C program.

## ■ Question: Why is this topic important?

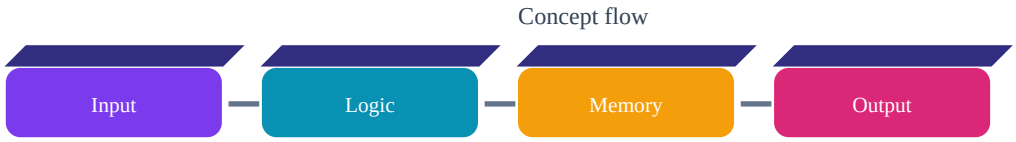
■ **Answer:** It is a foundation for writing, reading, debugging and explaining C programs.

## ■ Question: What should a beginner practice?

■ **Answer:** Write small programs, compile them, predict the output, and then run them.

# 7. Loops

Loops repeat statements. C provides for, while and do-while loops.




## ■ Key Points

• for groups initialization, condition and update

• while checks before each iteration

• do-while executes at least once

• break exits a loop

• continue skips an iteration

## ■ Example

```
for (int i = 1; i <= 5; i++) {
printf("%d ", i);
}
```

## ■ Explanation

Read the code from top to bottom. Identify the input, processing logic, variables used, and final output. Then type the program yourself and change one value to see how the result changes.

## ■ Question: What is the main idea of this chapter?

■ **Answer:** Understand the concept, its syntax, and where it is useful in a C program.

## ■ Question: Why is this topic important?

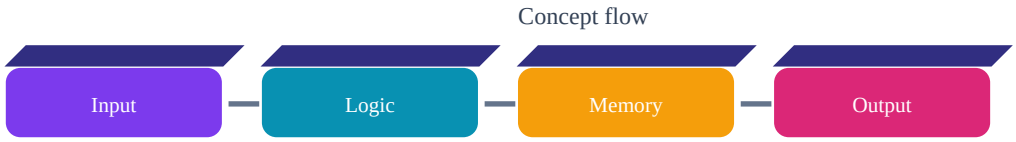
■ **Answer:** It is a foundation for writing, reading, debugging and explaining C programs.

## ■ Question: What should a beginner practice?

■ **Answer:** Write small programs, compile them, predict the output, and then run them.

# 8. Functions & Recursion

Functions divide programs into reusable units. Recursion occurs when a function calls itself.




## ■ Key Points

• Function prototype

• Function definition

• Function call

• Parameters

• Return value

• Base condition for recursion

## ■ Example

```
int add(int a, int b) {
return a + b;
}

int main(void) {
printf("%d", add(4, 5));
return 0;
}
```

## ■ Explanation

Read the code from top to bottom. Identify the input, processing logic, variables used, and final output. Then type the program yourself and change one value to see how the result changes.

## ■ Question: What is the main idea of this chapter?

■ **Answer:** Understand the concept, its syntax, and where it is useful in a C program.

## ■ Question: Why is this topic important?

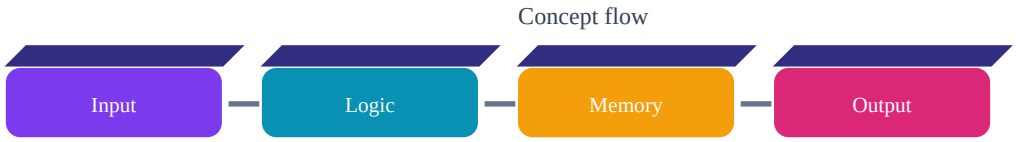
■ **Answer:** It is a foundation for writing, reading, debugging and explaining C programs.

## ■ Question: What should a beginner practice?

■ **Answer:** Write small programs, compile them, predict the output, and then run them.

# 9. Arrays

An array stores multiple values of the same type in contiguous memory. C uses zero-based indexing.




## ■ Key Points

• int a[5] creates five int elements

• First index is 0

• Loops traverse arrays

• 2D arrays represent tables/matrices

• Stay within valid bounds

## ■ Example

```
int marks[5] = {80,75,91,88,95};
for (int i=0; i<5; i++)
printf("%d ", marks[i]);
```

## ■ Explanation

Read the code from top to bottom. Identify the input, processing logic, variables used, and final output. Then type the program yourself and change one value to see how the result changes.

## ■ Question: What is the main idea of this chapter?

■ **Answer:** Understand the concept, its syntax, and where it is useful in a C program.

## ■ Question: Why is this topic important?

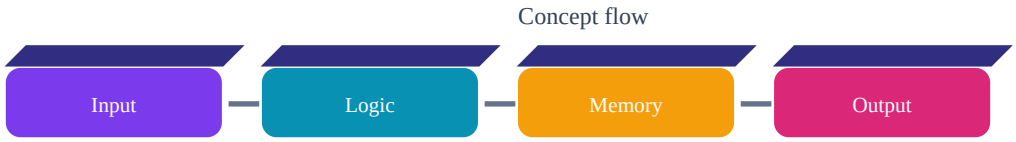
■ **Answer:** It is a foundation for writing, reading, debugging and explaining C programs.

## ■ Question: What should a beginner practice?

■ **Answer:** Write small programs, compile them, predict the output, and then run them.

# 10. Strings

A C string is a character array terminated by the null character \0.




## ■ Key Points

• char arrays store strings

• strlen() finds length

• strcpy() copies

• strcat() joins

• strcmp() compares

## ■ Example

```
#include
char name[] = "Vinay";
printf("%zu", strlen(name));
```

## ■ Explanation

Read the code from top to bottom. Identify the input, processing logic, variables used, and final output. Then type the program yourself and change one value to see how the result changes.

## ■ Question: What is the main idea of this chapter?

■ **Answer:** Understand the concept, its syntax, and where it is useful in a C program.

## ■ Question: Why is this topic important?

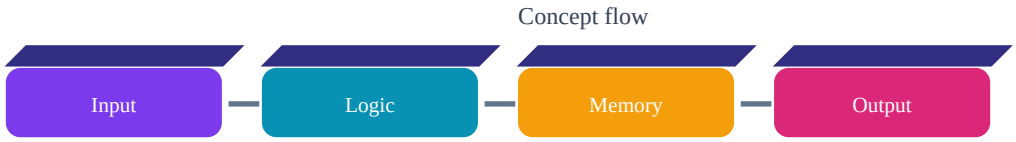
■ **Answer:** It is a foundation for writing, reading, debugging and explaining C programs.

## ■ Question: What should a beginner practice?

■ **Answer:** Write small programs, compile them, predict the output, and then run them.

# 11. Pointers

A pointer stores the address of another object. & obtains an address and * dereferences a pointer.




## ■ Key Points

• Address operator &

• Dereference operator *

• Pointer initialization

• Pointer arithmetic

• Pointers and arrays

## ■ Example

```
int x = 10;
int *p = &x;
printf("%d", *p);
```

## ■ Explanation

Read the code from top to bottom. Identify the input, processing logic, variables used, and final output. Then type the program yourself and change one value to see how the result changes.

## ■ Question: What is the main idea of this chapter?

■ **Answer:** Understand the concept, its syntax, and where it is useful in a C program.

## ■ Question: Why is this topic important?

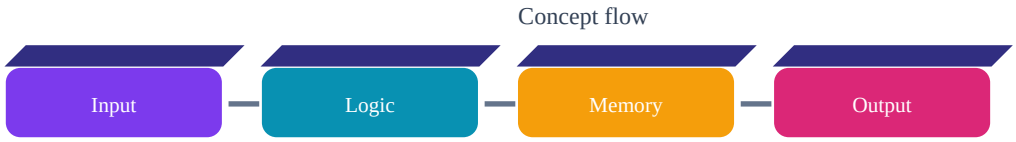
■ **Answer:** It is a foundation for writing, reading, debugging and explaining C programs.

## ■ Question: What should a beginner practice?

■ **Answer:** Write small programs, compile them, predict the output, and then run them.

# 12. Structures & Unions

A structure groups related members, possibly of different types. Union members share storage.




## ■ Key Points

• struct creates a structure type

• Dot operator accesses a member

• -> accesses through a structure pointer

• Union members share storage

## ■ Example

```
struct Student {
int roll;
char name[30];
float marks;
};
struct Student s = {1, "Aman", 88.5f};
```

## ■ Explanation

Read the code from top to bottom. Identify the input, processing logic, variables used, and final output. Then type the program yourself and change one value to see how the result changes.

## ■ Question: What is the main idea of this chapter?

■ **Answer:** Understand the concept, its syntax, and where it is useful in a C program.

## ■ Question: Why is this topic important?

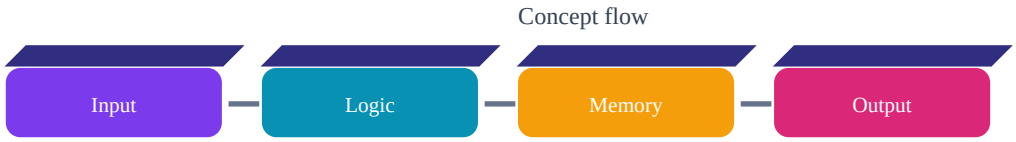
■ **Answer:** It is a foundation for writing, reading, debugging and explaining C programs.

## ■ Question: What should a beginner practice?

■ **Answer:** Write small programs, compile them, predict the output, and then run them.

# 13. File Handling

File handling lets programs store and retrieve information from files.




## ■ Key Points

• fopen() opens

• fclose() closes

• fprintf()/fscanf() formatted I/O

• fgets()/fputs() text I/O

• Check for NULL after fopen

## ■ Example

```
FILE *fp = fopen("notes.txt", "w");
if (fp != NULL) {
fprintf(fp, "BCA HUB");
fclose(fp);
}
```

## ■ Explanation

Read the code from top to bottom. Identify the input, processing logic, variables used, and final output. Then type the program yourself and change one value to see how the result changes.

## ■ Question: What is the main idea of this chapter?

■ **Answer:** Understand the concept, its syntax, and where it is useful in a C program.

## ■ Question: Why is this topic important?

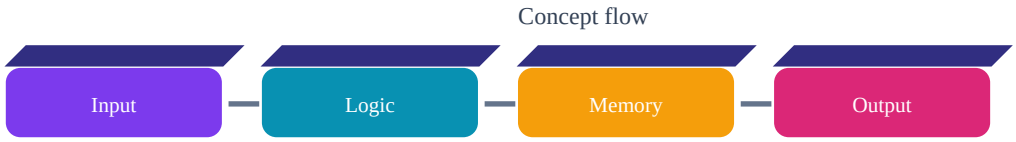
■ **Answer:** It is a foundation for writing, reading, debugging and explaining C programs.

## ■ Question: What should a beginner practice?

■ **Answer:** Write small programs, compile them, predict the output, and then run them.

# 14. Dynamic Memory

Dynamic memory is allocated at runtime. malloc, calloc, realloc and free are the key functions.




## ■ Key Points

• malloc allocates

• calloc allocates and zero-initializes

• realloc changes size

• free releases memory

• Check allocation results

## ■ Example

```
int *p = malloc(5 * sizeof(int));
if (p != NULL) {
p[0] = 10;
free(p);
p = NULL;
}
```

## ■ Explanation

Read the code from top to bottom. Identify the input, processing logic, variables used, and final output. Then type the program yourself and change one value to see how the result changes.

## ■ Question: What is the main idea of this chapter?

■ **Answer:** Understand the concept, its syntax, and where it is useful in a C program.

## ■ Question: Why is this topic important?

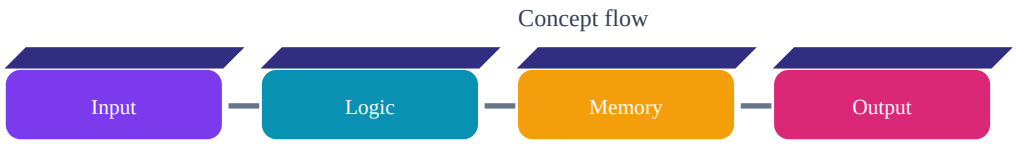
■ **Answer:** It is a foundation for writing, reading, debugging and explaining C programs.

## ■ Question: What should a beginner practice?

■ **Answer:** Write small programs, compile them, predict the output, and then run them.

# 15. Preprocessor & Header Files

The preprocessor handles directives before compilation. Header files provide reusable declarations.




## ■ Key Points

• #include

• #define

• conditional compilation

• header guards

• macros

## ■ Example

```
#include
#define PI 3.14159

int main(void) {
printf("%f", PI);
return 0;
}
```

## ■ Explanation

Read the code from top to bottom. Identify the input, processing logic, variables used, and final output. Then type the program yourself and change one value to see how the result changes.

## ■ Question: What is the main idea of this chapter?

■ **Answer:** Understand the concept, its syntax, and where it is useful in a C program.

## ■ Question: Why is this topic important?

■ **Answer:** It is a foundation for writing, reading, debugging and explaining C programs.

## ■ Question: What should a beginner practice?

■ **Answer:** Write small programs, compile them, predict the output, and then run them.

# 16. ■ Practice Programs

## Add Two Numbers

```
#include
int main(void) {
int a,b;
scanf("%d %d",&a;,&b;);
printf("Sum = %d",a+b);
return 0;
}
```

Practice: change the input/array values and predict the output before running.

## Even or Odd

```
#include
int main(void) {
int n;
scanf("%d",&n;);
if(n%2==0) printf("Even");
else printf("Odd");
return 0;
}
```

Practice: change the input/array values and predict the output before running.

## Factorial

```
#include
int main(void) {
int n,i;
long long f=1;
scanf("%d",&n;);
for(i=1;i<=n;i++) f*=i;
printf("%lld",f);
return 0;
}
```

Practice: change the input/array values and predict the output before running.

## Largest of Three

```
#include
int main(void) {
int a,b,c;
scanf("%d%d%d",&a;,&b;,&c;);
if(a>=b && a>=c) printf("%d",a);
else if(b>=a && b>=c) printf("%d",b);
else printf("%d",c);
return 0;
}
```

Practice: change the input/array values and predict the output before running.

## Reverse a Number

```
#include
int main(void) {
int n,rev=0,d;
scanf("%d",&n;);
while(n!=0) {
d=n%10;
rev=rev*10+d;
n/=10;
}
printf("%d",rev);
return 0;
}
```

Practice: change the input/array values and predict the output before running.

## Array Sum

```
#include
int main(void) {
int a[5]={10,20,30,40,50},i,sum=0;
for(i=0;i<5;i++) sum+=a[i];
printf("%d",sum);
return 0;
}
```

Practice: change the input/array values and predict the output before running.

# 17. ■ Important Questions & Answers

## ■ Question: Explain the basic structure of a C program.

■ **Answer:** A C program may contain comments, preprocessor directives, declarations, function definitions and main(). The statements inside functions perform the actual work. Header files provide declarations for library facilities.

## ■ Question: Differentiate while and do-while.

■ **Answer:** while checks its condition before the body and may execute zero times. do-while executes the body first and checks afterward, so it executes at least once.

## ■ Question: Explain an array.

■ **Answer:** An array is a collection of elements of the same type stored sequentially. It is accessed using an index beginning at zero and is commonly processed with loops.

## ■ Question: Explain a pointer.

■ **Answer:** A pointer stores an address. If int x=10 and int *p=&x;, p stores the address of x and *p accesses its value. Pointers are important for arrays, functions and dynamic memory.

## ■ Question: Explain functions.

■ **Answer:** A function is a reusable block of code. It can accept parameters and return a value. Functions make programs modular, easier to test and easier to maintain.

## ■ Question: Explain file handling.

■ **Answer:** File handling uses a FILE pointer and functions such as fopen and fclose. Programs can read or write data using formatted or line-based functions. Files should be checked for successful opening and closed after use.

# 18. ■ MCQs

**1. Usual entry point?**

A) start B) main C) run D) begin

**Answer: B) main**

**2. Remainder operator?**

A) / B) % C) // D) rem

**Answer: B) %**

**3. First array index?**

A) 0 B) 1 C) -1 D) 2

**Answer: A) 0**

**4. Header for printf?**

A) string.h B) stdio.h C) math.h D) time.h

**Answer: B) stdio.h**

**5. Address operator?**

A) * B) & C) # D) @

**Answer: B) &**

**6. Loop guaranteed to execute once?**

A) for B) while C) do-while D) none

**Answer: C) do-while**

**7. Equality comparison?**

A) = B) == C) := D) =>

**Answer: B) ==**

**8. Dynamic memory release?**

A) delete B) remove C) free D) clear

**Answer: C) free**

# 19. ■ Viva Questions

### 1. What is a compiler?

Answer tip: define it first, give one small example, then mention its use.

### 2. What is a variable?

Answer tip: define it first, give one small example, then mention its use.

### 3. What is a pointer?

Answer tip: define it first, give one small example, then mention its use.

### 4. What is recursion?

Answer tip: define it first, give one small example, then mention its use.

### 5. What is an array?

Answer tip: define it first, give one small example, then mention its use.

### 6. What is a string?

Answer tip: define it first, give one small example, then mention its use.

### 7. What is a structure?

Answer tip: define it first, give one small example, then mention its use.

### 8. What is NULL?

Answer tip: define it first, give one small example, then mention its use.

### 9. What is a format specifier?

Answer tip: define it first, give one small example, then mention its use.

### 10. Why close a file?

Answer tip: define it first, give one small example, then mention its use.

### 11. What is dynamic memory?

Answer tip: define it first, give one small example, then mention its use.

### 12. What is a header file?

Answer tip: define it first, give one small example, then mention its use.

# 20. ■ Final Revision Checklist

■ Program structure

■ Tokens and keywords

■ Data types and variables

■ printf/scanf

■ Operators

■ if/else and switch

■ Loops

■ Functions and recursion

■ Arrays

■ Strings

■ Pointers

■ Structures/unions

■ Files

■ Dynamic memory

■ Preprocessor

■ Practice 10+ programs

■ MCQs

■ Viva

**Best study method:** Don't only read. Type every program, compile it, predict the output, change one line, and run it again. This builds real programming skill.